
CONTRACTBENCH: Can LLM Agents Preserve Observation Contracts?

Jicheng Wang¹ Yifeng He^{1†} Zili Wang^{1†}
Hanwen Xing² Arkaprava De Hao Chen^{3*}

¹University of California, Davis ²University of Southern California

³University of Hong Kong

{jicwang, yfhe, zliwang}@ucdavis.edu, harvenx01@gmail.com,
arkaprava08@gmail.com, chenho@hku.hk

*Corresponding author. [†]Equal contribution.

Abstract

Tool-augmented LLM agents call APIs whose intermediate outputs, such as pre-signed URLs, session tokens, and OAuth state parameters, are *observation contracts*: artifacts whose later use is constrained by the external system that produced them. We show that observation contract compliance (preserving the temporal validity and byte-level integrity) is an emergent, regression-prone capability: it is neither guaranteed by general tool-use ability nor consistently improved by larger or newer models. To measure this, we introduce CONTRACTBENCH, a benchmark of 33 dual-axis tasks that probe two orthogonal failure modes no existing benchmark evaluates: validity failures (using an artifact after expiry) and integrity failures (corrupting an artifact’s bytes through the observation-to-action pipeline). Our evaluation is deterministic and programmatic, with a virtual clock controls time and SHA-256 hashes verify byte integrity. We assign the outcomes failure labels drawn from real-world API specifications. We evaluate 38 models and report four findings: (i) no evaluated model clears 80%, with Claude-Opus-4.6 leading at 77.8%, revealing that current frontier models still fail to comply with observation contracts. (ii) a sharp *within-family capability cliff* in Qwen 3.5 between 4B (0%) and 9B (56.6%), smoothing to 70.7% at 397B-A17B: what emerges across the cliff is mid-trajectory restraint, not tool-call competence; (iii) *non-monotonic scaling* across GPT-5 family: agentic post-training can erode compliance through sycophancy-driven regression; (iv) and our failure taxonomy works as an actionable in-context reward signal, yielding +7.1 pp on 42 paired GPT-5.1 failures.

1 Introduction

Large language model (LLM) agents are increasingly deployed as autonomous assistants by personal users and enterprises [Schick et al., 2023, Yao et al., 2023]. To reach a user-specified goal, an LLM agent calls external networked APIs, navigates websites, executes shell commands, and reads or writes files, observing the outcome of each step before choosing the next. These agents are also growing more autonomous and longer-lived, with recent systems completing everyday online tasks across dozens of live platforms with minimal human oversight [Zhang et al., 2026]. In production, they already resolve software-engineering issues [Yang et al., 2024], handle multi-turn customer-service dialogues [Yao et al., 2025], and carry out complex terminal tasks in scientific computing and cybersecurity [Merrill et al., 2026]. In this setting, a wrong action can break a workflow, corrupt a database, or cause an irreversible side effect [He et al., 2025]. Executing reliably matters as much as understanding the task.

Several benchmarks now ground agent evaluation in real APIs, codebases, and user interactions: web navigation [Zhou et al., 2024], software engineering [Jimenez et al., 2024, Merrill et al., 2026], general tool use [Qin et al., 2024, Liu et al., 2024], user interaction under domain-specific policies [Yao et al., 2025], and multi-constraint planning [Xie et al., 2024]. In these benchmarks, the intermediate steps of a multi-step task do not impose constraints on the steps that follow: a tool output is information for the agent’s next decision, and the agent can paraphrase, summarize, or even lose it without penalty so long as the workflow reaches the right end state. Real API workflows behave differently. The output of one step often dictates exactly how the agent must execute the next step. For example, the agent must relay a signed URL byte-for-byte, use a token before it expires, and round-trip an OAuth state parameter unmodified. Current benchmarks do not test whether the agent respects these inter-step constraints. Yet this is a primary failure mode in deployment: a single intermediate-step violation invalidates the whole task flow.

Previous AI agent evaluations have overlooked this setting, but it is routine in enterprise API workflows: a tool returns an artifact, and the external system that produced it constrains its later use. We call such a tool-returned artifact an *observation contract*. Consider a presigned S3 URL returned by a cloud-storage API. The URL works only before its expiration time and only if the agent preserves its signature-bound query string byte-for-byte [Amazon Web Services, 2026]. An agent can understand the user’s goal, call the right API, and still fail the workflow by using the URL too late, truncating it, re-encoding reserved characters, or reordering signed parameters. This failure pattern is not unique to presigned URLs. OAuth state parameters, session tokens, webhooks authenticated with HMAC (hash-based message authentication code), and rate-limit windows all impose the same two constraints: *temporal validity* (use the artifact before it expires or becomes stale) and *byte-level integrity* (relay the artifact without alteration). The two constraints are orthogonal. Refreshing an expired artifact repairs validity but does not prevent in-transit corruption of every fresh copy; preserving bytes intact does not teach the agent to respect deadlines or version freshness.

Existing benchmarks partly address the temporal axis and largely overlook the integrity axis. Table 1 surveys the landscape: several include partial temporal constraints such as time limits or session windows, and recent work on temporal blindness shows that agents can mishandle stale information [Cheng et al., 2026]. None, however, jointly tests temporal validity and byte-level artifact integrity. In practice, expired tokens, mutated URLs, stale ETags (HTTP entity tags), and signature mismatches break otherwise correct workflows.

Table 1: Agent benchmark landscape. “V”: temporal validity; “I”: byte-level integrity; “P”: programmatic evaluation (no human or LLM judge).

Benchmark	V	I	P
SWE-bench [Jimenez et al., 2024]	✗	✗	✓
ToolBench [Qin et al., 2024]	✗	✗	Partial
AgentBench [Liu et al., 2024]	✗	✗	✓
TravelPlanner [Xie et al., 2024]	✗	✗	Partial
τ -bench [Yao et al., 2025]	✗	✗	✓
Terminal-Bench [Merrill et al., 2026]	✗	✗	✓
TicToc [Cheng et al., 2026]	Partial	✗	✓
CONTRACTBENCH (Ours)	✓	✓	✓

To address this gap, we introduce CONTRACTBENCH, a benchmark for measuring observation-contract compliance in LLM agents. CONTRACTBENCH contains 33 tasks drawn from real API contract patterns — presigned URLs, OAuth state parameters, signed requests, HMAC-protected webhooks, rate-limit windows, and multi-step token workflows — arranged across the two orthogonal axes and eight contract domains, shown in Figure 1. Every task runs under a deterministic virtual clock, validates artifacts programmatically, and assigns each failed episode one primary label from a taxonomy of 15 failure modes that map directly onto either the validity or the integrity axis. The benchmark measures both whether an agent reaches a goal and whether it preserves the contracts attached to intermediate tool outputs. Across 38 model variants from 15 families, observation-contract compliance is unsolved at the frontier (best model 77.8 % on the full 33-task suite, $n=99$) and behaves non-monotonically with scale and post-training: a sharp within-family capability cliff in Qwen 3.5 and a V-shape across GPT-5 generations at near-constant parameter count. Our contributions are:

1. **Observation contracts.** We formalize observation contracts as tool-returned artifacts governed by two orthogonal constraints, temporal validity and byte-level integrity, and define a taxonomy of 15 failure modes for diagnosing contract failures.
2. **CONTRACTBENCH.** We build a deterministic benchmark of 33 agent tasks drawn from real API contract patterns, covering validity-dominant, integrity-dominant, and dual-axis pressures. Each

task has a programmatic validator, a reproducible virtual-clock execution model, and a reference solution that establishes solvability.

3. **Empirical findings.** Across 38 model variants spanning 15 families, contract compliance is neither guaranteed by general tool-use ability nor consistently improved by larger or newer models. We observe a sharp within-family capability cliff in Qwen 3.5 (0% at $\leq 4B$ to 70.7% at 397B-A17B), a GPT-version regression and recovery (23% $\rightarrow$ 71% $\rightarrow$ 49% $\rightarrow$ 75%, full 33-task suite, $n=99$), and a structured failure hierarchy from protocol entry to long-horizon constraint tracking.

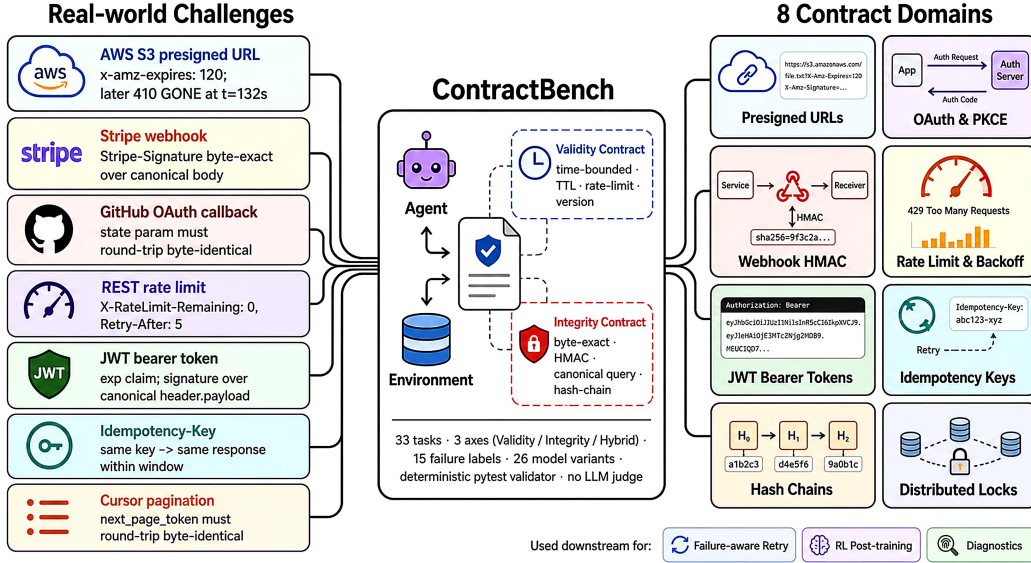


Figure 1: CONTRACTBENCH is the first deterministic benchmark for LLM-agent observation-contract compliance, spanning 33 tasks across two orthogonal axes (temporal validity and byte-level integrity) and eight real-world API contract domains.

2 Observation Contracts

Production agents repeatedly hit the same class of failure: a tool returns an *artifact* (an intermediate value such as a presigned URL, a signed token, or an OAuth state parameter that the agent must reuse in a later step) that carries a time-limit and a byte-level integrity rule, and a downstream tool call rejects it because one of those rules was violated somewhere along the way [Amazon Web Services, 2026]. We call this an *observation contract* failure, which we adopt from the enforcement clauses of widely deployed API specifications (AWS SigV4, OAuth 2.0, JWT, Stripe webhooks, RFC 6585), where every contract clause we observed reduces to a temporal check, a byte-level check, or a conjunction of the two. Despite its prevalence in deployed agent stacks, the agent-evaluation literature has not formalized this class or measured agents against it directly. We provide a formal definition of the problem in this section, and introduce CONTRACTBENCH in Section 3.

Let $\mathcal{O}$ be an artifact space (e.g., the set of UTF-8 strings), $\mathcal{T} \subseteq \mathbb{R}_{\geq 0}$ a virtual time domain, and $\mathcal{H}$ a digest space (in our implementation, $\mathcal{H} = \{0, 1\}^{256}$ via SHA-256).

Definition 2.1 (Observation contract). An observation contract is a 4-tuple $C = (o, t_{\text{issue}}, \tau, \pi)$, where $o \in \mathcal{O}$ is the artifact issued at time $t_{\text{issue}} \in \mathcal{T}$, $\tau \in \mathbb{R}_{>0}$ is its time-to-live, and $\pi : \mathcal{O} \rightarrow \{0, 1\}$ is an integrity predicate. The induced validity window is $W(C) := [t_{\text{issue}}, t_{\text{issue}} + \tau] \subseteq \mathcal{T}$.

In our implementation π is the SHA-256 equality check $\pi(o') = \mathbb{1}[\text{SHA-256}(o') = h(o)]$, but the framework admits any deterministic predicate (HMAC, ETag equality, JWT signature verification).

Definition 2.2 (Satisfaction). A submission pair $(o', t') \in \mathcal{O} \times \mathcal{T}$ satisfies a contract C if and only if

$$\underbrace{t' \in W(C)}_{\text{validity}} \quad \text{and} \quad \underbrace{\pi(o') = 1}_{\text{integrity}}.$$

We write $\text{Sat}(C, o', t') \in \{0, 1\}$ for the satisfaction indicator.

Definition 2.3 (Failure modes). A submission (o', t') is a validity failure on C when $t' \notin W(C)$, and an integrity failure on C when $\pi(o') = 0$. The two modes are logically independent: a submission may exhibit one, both, or neither.

Definition 2.4 (Compliance). A trace of n contracts $\{C_i\}_{i=1}^n$ with submissions $\{(o'_i, t'_i)\}_{i=1}^n$ is contract-compliant if and only if

$$\text{Compliant}(\{C_i\}, \{(o'_i, t'_i)\}) := \prod_{i=1}^n \text{Sat}(C_i, o'_i, t'_i) = 1.$$

Equivalently, the trace is non-compliant whenever $\text{Sat}(C_i, o'_i, t'_i) = 0$ for some $i \in \{1, \dots, n\}$.

Proposition 1 (Orthogonality). For any contract C with non-empty validity window and non-trivial integrity predicate, all four cells of the 2×2 validity-by-integrity partition over submissions are non-empty (Figure 2). Consequently, fixing one axis (e.g., always submitting at $t' \in W$) does not fix the other. (Proof sketch in Appendix I.)

The proposition is a structural claim: the two axes can vary independently in principle. The substantive claim is that they vary independently in practice, in real LLM agent stacks. Refreshing an expired URL repairs validity but does not stop a downstream pipeline from corrupting the fresh copy through truncation, re-encoding, or parameter reordering. Conversely, preserving bytes intact does not teach an agent to respect deadlines or rate-limit windows. Section 4 confirms this empirically: models near the frontier on one axis collapse on the other. This independence is what makes the problem worth a dedicated benchmark: a single-axis evaluation cannot expose it. The next section describes how CONTRACTBENCH probes both axes, separately and jointly.

3 CONTRACTBENCH

We introduce CONTRACTBENCH, a benchmark that instantiates the observation-contract problem (Section 2) as a suite of programmatically evaluated tasks. CONTRACTBENCH contains 33 tasks, all generated from parameterized templates, evaluated by deterministic validators, and executed under a virtual clock for full reproducibility. This section describes the dual-axis task design (Section 3.1) following Definition 2.2, and the deterministic evaluation protocol (Section 3.2).

3.1 Building Tasks that Probe Both Axes Simultaneously

Two-level taxonomy. We organize tasks using a two-level taxonomy, as depicted in Figure 2. The top level groups tasks by primary pressure quadrant following the orthogonality of Proposition 1: **Q1** (low validity, low integrity) is a sanity check; **Q2** isolates temporal planning; **Q3** isolates byte preservation; and **Q4** stresses both axes simultaneously. The Q4-heavy distribution (24 of 33 tasks) reflects the real-world prevalence of artifacts that are both time-limited and signature-bound. The second level (visible inside Q4) groups dual-axis tasks by *contract pattern*: the real-world API pattern being tested (OAuth / auth flows, signed requests, state chains, resource management, multi-service flows). Per-task counts and one example task per pattern are shown in the figure; the full per-task list is in Appendix D.

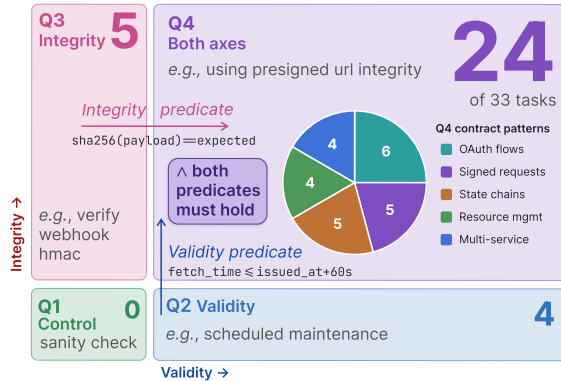


Figure 2: **Two orthogonal predicates define an observation contract.** Integrity ($\text{SHA-256}(\text{payload}) = \text{expected}$) and validity ($t_{\text{fetch}} \leq t_{\text{issue}} + \tau$) must *both* hold (Definition 2.2); the four quadrants instantiate Proposition 1. Quadrant area encodes CONTRACTBENCH’s task distribution.

Task structure. Each CONTRACTBENCH task consists of four files: 1. a TOML metadata header that fixes difficulty, dual-axis category, and timing budgets; 2. a Markdown instruction visible to the

Table 2: Failure taxonomy (15 labels). Each failed episode receives one primary label, aggregated from the underlying event list using the most-severe rule (Appendix K).

Group	Label	Description
Validity	EXPIRED_BEFORE_USE	Resource TTL expired before agent used it [Jones and Hardt, 2012].
	RATE_LIMITED	Exceeded API rate-limit window (HTTP 429) [Nottingham and Fielding, 2012].
	SCHEDULED_UNAVAILABLE	Called service during scheduled downtime (HTTP 503) [Fielding et al., 2022].
	VERSION_CONFLICT	Submitted write with stale ETag / If-Match [Fielding and Reschke, 2014].
Integrity	WRONG_VALUE	Required field present but value incorrect (benchmark-internal predicate; see Section 3.1).
	MISSING_CONSTRAINT	Required protocol field omitted from request [Hardt, 2012, §4.1.1].
	MUTATED_TOKEN	Bytes altered by URI normalization/re-encoding /reordering [Berners-Lee et al., 2005, §6].
	SIGNATURE_MISMATCH	Request signature failed verification [Backman et al., 2024, Amazon Web Services, 2024].
	COMPENSATION_FAILURE	Saga compensation step did not undo prior write [Garcia-Molina and Salem, 1987].
	WRONG_HASH	Content digest computed incorrectly [NIST, 2015, Polli and Pardue, 2024].
	SHORTCUT_TAKEN	Adversarial shortcut endpoint used instead of canonical flow [Krakovna et al., 2020].
	MISSING_TOKEN	Required authentication token absent at submission [Hardt, 2012, §4].
Meta	BACKOFF_VIOLATION	Retried before <code>Retry-After</code> window elapsed [Fielding et al., 2022, §10.2.3].
	SUCCESS	Contract satisfied on both axes.
	OTHER	Unclassified or low-frequency variant (catch-all for emitted strings outside the taxonomy).

agent; 3. a FastAPI server that emits one of 15 deterministic failure labels per HTTP request via a shared `append_log()` call running over a virtual clock; 4. a `pytest` validator that consumes the request log and computes the reward. The agent only sees the instruction and the server’s HTTP responses; the metadata header and validator are hidden, eliminating test-set leakage and LLM-as-judge ambiguity. We build CONTRACTBENCH on Harbor [Harbor Framework Team, 2026] following standard practice of agent evaluation [Merrill et al., 2026, Li et al., 2026]. We provide an annotated example in Appendix Figure 5 to illustrate the task structure.

Failure taxonomy. CONTRACTBENCH instantiates the abstract validity/integrity axes as a 15-label taxonomy (Table 2), partitioned into four *validity* failures (temporal), nine *integrity* failures (byte-level), and two *meta* labels. Each label in Table 2 traces to a documented real-world API rule. We provide the full per-task event-to-label mapping in Appendix K. At runtime, each task server emits a chronological list of violation events into the request log; the validator aggregates these events into a single *primary label* per episode using the most-severe-label rule, and protocol state events that appear in successful episodes (e.g., CHALLENGE_ISSUED) are excluded from the failure distribution.

Two sources of difficulty stack in CONTRACTBENCH. First, the path from API response, through the agent’s context window, into the next tool call is not byte-preserving in production: tool-call interfaces truncate long inputs, HTTP libraries re-encode percent-escaped URLs, middleware re-sorts query parameters before signing, and rendered link text disagrees with the underlying href. These mutations are not artifacts we invent: they arise naturally from the way real agent stacks compose. CONTRACTBENCH reproduces five of them as deterministic toggles so each cause can be isolated, and the full catalog and the failure label each one triggers is in Appendix J. Furthermore, even without mutations, dual-axis tasks force the agent to plan around tight TTLs while preserving exact byte content. Appendix H walks through one validity-heavy and one integrity-heavy task that exemplify the two pressures and the corresponding agent-side strategies.

Task generation and solvability. Each task is instantiated from a parameterized template that fixes the contract type, the difficulty knobs (TTL duration, rate-limit window, token length), and the validator. A single *seed* controls every random element (resource ordering, token content, timing jitter), so a given (template, seed) pair always produces the same task instance, and we evaluate each model on a fixed grid of 10 seeds per task. Following the verification practice of Terminal-bench, every CONTRACTBENCH task ships with an *oracle solution* that issues the correct sequence of API calls under the same virtual clock as the agent. We require each oracle to pass the validator before admitting the task; all 33 oracles do so on every seed ($33 \times 10 = 330$ oracle episodes), so every task is provably solvable and the validator accepts the intended behavior. Oracle solutions serve as regression tests during task authoring and reproducibility checks. They are not exposed to the agent during model evaluation.

3.2 Evaluation Protocol

CONTRACTBENCH uses deterministic, programmatic validators for reliability: no LLM-as-judge, no human raters. The validator consumes the per-episode HTTP request log written by the task server and outputs the reward details: a binary success flag, the most-severe failure label drawn from the taxonomy (Table 2), the failure detail, and trace metadata (Appendix G).

Metrics. Each model is evaluated at $k=3$ runs per task across 33 tasks ($n=99$ episodes per model). We report three quantities derived from the validator output. **Success rate** (SR %) is the fraction of the $n=99$ episodes with label SUCCESS. **Per-task pass rate** is the mean reward across the $k=3$ runs of one model on one task. **Failure-label distribution** is the distribution of primary labels over a model’s failed episodes (counting convention in Appendix L).

Success criteria. An episode is labeled SUCCESS if and only if its HTTP request log satisfies the contract per Definition 2.2: every required observation is both temporally valid ($t_{\text{fetch}} \leq t_{\text{issue}} + \tau$) and byte-integral ($\text{SHA-256}(\text{payload}) = \text{expected}$); any other event in the log triggers the most-severe failure label. The validator reads HTTP-level events only, never agent prose or self-reported summaries, so an agent cannot bluff its way to success by narrating compliance it did not perform.

4 Experiments

Models and settings. We evaluate 38 model variants across 15 families, grouped into three categories: (i) frontier proprietary models (Claude, GPT-5 series, and Gemini 2.5); (ii) open-source instruction-tuned models (Qwen 3.5, Qwen 2.5, MiniMax, Mistral / Ministral, Gemma, Llama, DeepSeek-R1) spanning a range of parameter counts and training recipes; (iii) Base (non-instruction-tuned) and smaller Instruct checkpoints, included to disentangle parameter count from post-training. Per-model provider and infrastructure metadata, plus the full 38-row leaderboard, are in Appendix Table 10. Each model is run on the full 33-task CONTRACTBENCH suite at $k=3$ rollouts per task ($n=99$ episodes per model), with temperature 0, a task-describing system prompt, and a 600-second per-episode wall-clock timeout.

Research questions. To probe how contract compliance varies with models, scale, post-training, and failure mode, we ask four questions. **RQ1** how do frontier models perform on CONTRACTBENCH? **RQ2** how does compliance scale with parameters within the same training method, and does the within-family pattern reproduce across families? **RQ3** at approximately constant parameter count, can post-training updates change compliance, and if so, on which capability axes? **RQ4** does CONTRACTBENCH reward signal lead to improvement of the agents?

4.1 Frontier Models on Observation Contracts

The CONTRACTBENCH leaderboard (Table 3) places Claude Opus 4.6 at the top (77.8 %, 77/99); no evaluated model clears 80 %, so roughly one in four episodes still violates the contract even for the top model (per-task ceiling in Section 4.4). Frontier proprietary scores span 19.2 %–77.8 % (a 58.6-pp within-tier spread), and the best-open vs. best-proprietary gap is only 7.1 pp: Qwen3.5-397B-A17B matches GPT-5 at 70.7 %. Rank on CONTRACTBENCH does not track rank on general-purpose benchmarks — Gemini 2.5 Flash and GPT-5.1 sit below several open-source SOTA models, while Llama-3.3-70B (7.1 %) trails far behind same-class proprietary models. The GPT-5 series moves non-monotonically across versions (Section 4.3), and the open-source distribution falls off steeply at the bottom (Section 4.2).

Table 3: The CONTRACTBENCH leaderboard. $k=3$, $n=99$ episodes per model.

Model	Pass	SR%
Claude Opus 4.6	77	77.8
Claude Sonnet 4.5	69	69.7
GPT-5.2	74	74.8
GPT-5	70	70.7
GPT-5.1	48	48.5
GPT-4o	23	23.2
Gemini 2.5 Pro	51	51.5
Gemini 2.5 Flash	19	19.2
Qwen3.5-397B-A17B (MoE)	70	70.7
Qwen3.5-27B	64	64.6
Qwen3.5-9B	56	56.6
Qwen2.5-72B-Instruct	23	23.2
MiniMax-M2.5	62	62.6
MiniMax-M2.1	60	60.6
MiniMax-M2	53	53.5
Mistral-Small-4 (MoE)	42	42.4
Ministral-3-14B	28	28.3
Gemma-4-26B-A4B (MoE)	38	38.4
Gemma-4-31B	37	37.4
Llama-3.3-70B-Instruct	7	7.1

Finding 1: Frontier ceiling and cross-provider disagreement

No evaluated model clears 80 %, and frontier proprietary scores span 19.2 %–77.8 % (Claude Opus 4.6 leads). Rank on CONTRACTBENCH does not track general-purpose benchmark rank: Gemini 2.5 Flash and GPT-5.1 sit below several open-source SOTA models, and the 7.1-pp best-open vs. best-proprietary gap is narrower than the within-proprietary spread.

4.2 Within-Family Scaling: A Capability Cliff

The Qwen 3.5 cliff. We analyze the impact of scaling on observation contracts with the Qwen 3.5 dense family in Figure 3, and observe a sharp jumps on success rate between 4 B and 9 B. Below the cliff, the 4 B agent emits valid tool calls but gives up after one or two failed attempts, so every episode terminates before any contract predicate fires (100 % OTHER, a pre-contract failure rather than a taxonomy gap). Above it, traces show adaptive behavior: agents pivot between `curl` and `python3 urllib`, wait through scheduled maintenance, and back off after rate limits, and their residual failures move into frontier-style labels (MISSING_CONSTRAINT, temporal). What comes online is mid-trajectory restraint, not tool-call emission.

Other families do not reproduce the cliff. Qwen 2.5 Instruct climbs gradually instead of jumping (0 % at 1.5–32 B, 23.2 % only at 72 B). Mistral / Ministral rises smoothly (Ministral-3-3B/8B/14B at 6.1/19.2/28.3 %; Mistral-Small-4 MoE 42.4 %). Gemma 4 ramps early and then plateaus (E2B 7.1 %, E4B 17.1 %, 26B-A4B 38.4 %, 31B 37.4 %); E4B and 26B-A4B share 4 B active compute, so the dense-vs-MoE gap reads as a vertical jump in the panel. Phi-4 14 B Base, Llama-3.3-70B-Instruct, and DeepSeek-R1 671 B/37 B (Thinking MoE) all sit at or near 0 %, so chain-of-thought training alone does not lift compliance. The abruptness inside Qwen 3.5 fits the emergent-capability pattern of Wei et al. [2022], but the threshold tracks training recipe more than scale.

Finding 2: Capability cliff: emergence is family-specific, not parameter-fixed

Qwen 3.5 Instruct moves from 0 % at 4 B to 56.6 % at 9 B and scales smoothly to 70.7 % at 397 B-A17 B. The cliff localizes to the Qwen 3.5 training recipe rather than parameter count, and what turns on across it is adaptive recovery, not tool-call emission.

Contract compliance emerges from post-training. The Base→Instruct contrast in Figure 3 is consistent across the families for which Base weights are publicly available. Qwen 2.5 Base scores 0 % across 1.5–72 B and Mistral / Ministral Base does the same across 3–14 B; the Qwen 3.5 9 B Base→9 B Instruct comparison reproduces the gap inside the recipe that drives the cliff (Phi-4 14 B Base also at 0 %). We could not run the matching test for Qwen 3.5 27 B / 397 B-A17 B or Gemma 4, since their Base checkpoints are not publicly released. Wherever the comparison is possible, no Base checkpoint clears the floor while the matched Instruct version does. Pretraining scale alone is therefore insufficient: contract compliance is acquired during post-training. Section 4.3 shows the converse direction, that post-training updates at fixed parameter count can also erode this capability.

Finding 3: Contract compliance is a post-training capability

Frontier-tier compliance requires reasoning post-training or at least instruction-tuning on top of the base model. Pretraining scale alone does not produce contract compliance, and (Section 4.3) post-training can also remove it.

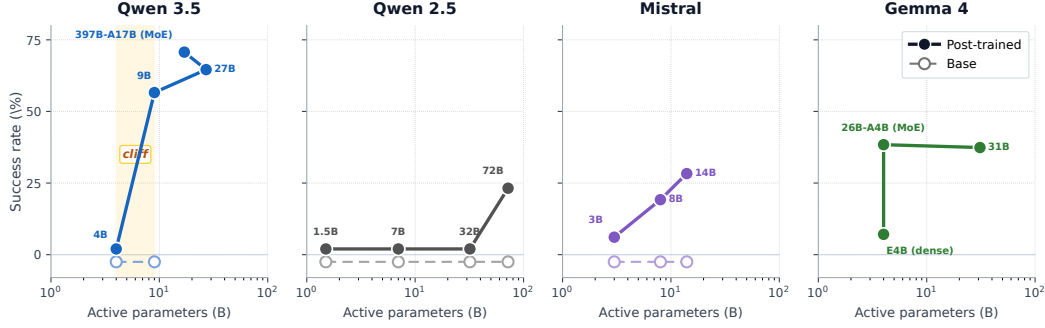


Figure 3: Within-family scaling on CONTRACTBENCH: at fixed parameter count, the Base→Instruct gap (post-training delta) is family-specific, with Qwen 3.5 reaching 56.6 % already at 9 B-Instruct.

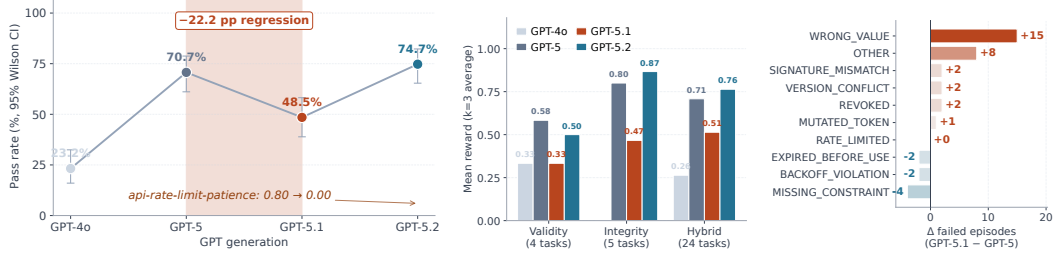
4.3 Post-Training Regression: A V-Shape in GPT-5

In Finding 3 we observe that contract compliance emerges from post-training. Holding the pretrained base fixed, we further investigate the converse question: how does scaling post-training on the same base affect contract compliance? We experiment on the GPT-5 series, where GPT-5, GPT-5.1, and GPT-5.2 share a pretrained base and differ only in post-training updates [OpenAI, 2025b,c, 2026b]. Across these generations, compliance on the full 33-task suite is non-monotonic (Figure 4a). We observe a *version regression* on GPT-5.1 driven entirely by post-training update choices, not model size or pretraining changes.

The regression is structured and locatable. GPT-5.1’s added failures are not spread evenly across failure types: they concentrate on the integrity axis, re-introducing modes that GPT-5 had largely suppressed (Figure 4b, Figure 4c). GPT-5.2 does not retrace this path; it lifts above GPT-5 with a more temporal mix of residual failures, so the V has a direction as well as a depth. The trajectory on `api-rate-limit-patience` shows an inverse-scaling (GPT-4o 0.80 → GPT-5 0.00). Together with the cliff (Section 4.2), this tells us what emerges and what regresses are specific *inhibitions*: waiting before retrying, copying bytes without paraphrasing, refusing when forbidden. Inhibitions are fragile: post-training can erode them while general benchmarks improve (Appendix A).

Finding 4: Version regression: post-training may erode byte-level integrity

At base model, the GPT-5 series exhibits a deep V-shape contract-compliance regression across post-training updates. The regression is structured: GPT-5.1’s failures concentrate on byte-level integrity, and post-training appears to erode the specific inhibitions (wait, byte-preserve, refuse) that contract compliance requires.



(a) V-trajectory across four GPT generations. (b) Per-family mean reward. (c) Failure label differences.

Figure 4: A V-shape performance within the GPT-5 family with shared pretrained base: a structured, locatable post-training version regression. The regression is structured (specific failure modes return) and locatable (concentrated on the integrity axis), not a uniform capability loss.

4.4 Failure Labels as an Actionable Reward Signal

We adapt test-time in-context reinforcement learning [Shinn et al., 2023] to test whether CONTRACTBENCH failure labels function as an actionable reward signal, focusing on whether label *content*, not the act of retrying, drives recovery. We apply CONTRACTBENCH’s failure labels as feedback on the 42 GPT-5.1 failures from Section 4.3, whose integrity-heavy profile (Appendix T) admits clean re-attempts. We experiment with three variants: *naive* (retry with no hint), *correct-label* (the original label injected as a coaching note), and *wrong-label* (a structurally-different far-axis label as a specificity control).

Table 4: Effects of in-context reinforcement using CONTRACTBENCH failure labels.

Retry condition	Pass	Rate	Δ vs naive
No hint	6 / 42	14.3 %	n/a
Wrong-label hint	5 / 42	11.9 %	−2.4 pp
Correct-label hint	8 / 42	19.0 %	+4.8 pp
Δ correct − wrong			+7.1 pp

Where context-level feedback suffices, and where it does not. We present the results of verbal reinforcement with failure labels in Table 4. Correct signal beats naive by +4.8 pp, while wrong signal *underperforms* naive by 2.4 pp; the headline gap Δ (correct−wrong) = +7.1 pp isolates label *content* as the operative variable, not the act of retrying. The labels therefore carry directional, label-specific corrective signal in the construct-validity sense of Bean et al. [2025] and Alaa et al. [2025]. We present the severity-weighted label-to-deficit mapping in Appendix K as a natural starting point. The +7.1 pp gap is not uniform across labels (full breakdown in Appendix Table 13): recovery concentrates in integrity-style labels that admit a clean re-attempt, while temporal and integrity-fragile labels are unresponsive, since a label that says “you waited too long” does not give the agent a recoverable next action. The single RATE_LIMITED case inverts: the *wrong-label* hint succeeds while the correct one does not, because telling a rate-limited agent “you were rate-limited” encourages further aggressive retries. Such failures point to architectural intervention (machine-readable TTLs, back-off middleware) rather than test-time signal.

Finding 5: CONTRACTBENCH labels are an actionable reward signal

CONTRACTBENCH’s failure labels carry directional, label-specific corrective signal: correct-label signal beats wrong-label coaching by +7.1 pp on 42 paired GPT-5.1 failures, a viable starting point for reward modeling and RL post-training. The signal splits by axis: integrity-style failures are fixable with in-context signals, whereas temporal failures require further efforts.

5 Related Work

Agent benchmarks. Existing benchmarks evaluate task completion (navigating websites [Zhou et al., 2024], resolving software issues [Jimenez et al., 2024], planning trips [Xie et al., 2024], or using tools [Qin et al., 2024, Liu et al., 2024]), but none test whether agents preserve the *artifacts* that tools return (Table 1). The closest work is TicToc [Cheng et al., 2026], which identifies “temporal blindness” (failure to re-invoke tools when cached information becomes stale). CONTRACTBENCH

extends this in two ways: we test *proactive* temporal planning (reordering by deadline), not just reactive staleness detection, and we add the orthogonal integrity axis.

Emergent capabilities and inverse scaling. Wei et al. [2022] observe that certain capabilities emerge abruptly at specific model scales rather than improving gradually. McKenzie et al. [2023] document *inverse scaling* (tasks where larger models perform worse), attributing this to overconfident pattern-matching. Our findings contribute to both literatures: the capability cliff (0% at 4B $\rightarrow$ 76.5% at 27B) demonstrates emergence, while `api-rate-limit-patience` (where capable models score *lower* due to aggressive retries) demonstrates inverse scaling in the agentic setting.

Iterative refinement and self-correction. Reflexion [Shinn et al., 2023] and Self-Refine [Madaan et al., 2023] retry failed episodes using the model’s own verbal self-critique as the corrective signal. Our predictive-validity experiment (Section 4.4) replaces self-critique with a deterministic, server-emitted failure label and adds a paired wrong-label control to isolate label *content* as the operative variable. CONTRACTBENCH’s 15-label taxonomy is also a natural process-reward signal for agentic RL post-training [Cheng et al., 2025, Pan et al., 2026], which we leave to future work.

6 Conclusion

We formalize *observation contracts*, a tool-returned artifact governed by a temporal validity window and a byte-level integrity predicate. We present CONTRACTBENCH, a deterministic benchmark for measuring observation contract compliance in LLM agents. CONTRACTBENCH comprises 33 tasks drawn from real API contract patterns, and probes the two orthogonal evaluation axes of observation contracts that prior agent benchmarks do not jointly evaluate. Each episode runs under a virtual clock with a programmatic validator and yields a structured failure label, which also serves as a signal for test-time correction and training. Our experiments show that contract compliance is unsolved at the frontier and behaves non-monotonically with scale and post-training. CONTRACTBENCH aims to make observation contracts a first-class evaluation target and to motivate further research on agents that act reliably under the inter-step constraints imposed by real external systems and APIs. Our source code and dataset are available on GitHub and Hugging Face, respectively.

References

- Ahmed Alaa, Thomas Hartvigsen, Niloufar Golchini, Shiladitya Dutta, Frances Dean, Inioluwa Deborah Raji, and Travis Zack. Position: Medical large language model benchmarks should prioritize construct validity, 2025. URL <https://arxiv.org/abs/2503.10694>.
- Amazon Web Services. AWS signature version 4 (SigV4) signing process. Amazon Web Services Documentation, 2024. URL https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_sigv.html.
- Amazon Web Services. AWS Prescriptive Guidance: Establishing guardrails and monitoring for pre-signed urls, 2026. URL <https://docs.aws.amazon.com/pdfs/prescriptive-guidance/latest/presigned-url-best-practices/presigned-url-best-practices.pdf#overview>.
- Anthropic. Claude Sonnet 4.5: Model card, 2025a. URL <https://www.anthropic.com/news/claude-sonnet-4-5>. Pinned identifier: `claude-sonnet-4-5-20250929`.
- Anthropic. Claude Opus 4.6: Model card and system card, 2025b. URL <https://www.anthropic.com/news/claude-opus-4-6>. Pinned identifier: `claude-opus-4-6`.
- A. Backman, J. Richer, and M. Sporny. HTTP message signatures. Request for Comments RFC 9421, Internet Engineering Task Force, February 2024. URL <https://www.rfc-editor.org/rfc/rfc9421>.
- Andrew M. Bean, Ryan Othniel Kearns, Angelika Romanou, Franziska Sofia Hafner, Harry Mayne, Jan Batzner, Negar Foroutan, Chris Schmitz, Karolina Korgul, Hunar Batra, Oishi Deb, Emma Beharry, Cornelius Emde, Thomas Foster, Anna Gausen, Maria Grandury, Simeng Han, Valentin

- Hofmann, Lujain Ibrahim, Hazel Kim, Hannah Rose Kirk, Fangru Lin, Gabrielle Kaili-May Liu, Lennart Luetzgau, Jabez Magomere, Jonathan Rystrom, Anna Sotnikova, Yushi Yang, Yilun Zhao, Adel Bibi, Antoine Bosselut, Ronald Clark, Arman Cohan, Jakob N. Foerster, Yarin Gal, Scott A. Hale, Inioluwa Deborah Raji, Christopher Summerfield, Philip H. S. Torr, Cozmin Ududec, Luc Rocher, and Adam Mahdi. Measuring what matters: Construct validity in large language model benchmarks, 2025. URL <https://arxiv.org/abs/2511.04703>. NeurIPS 2025 Datasets and Benchmarks Track.
- T. Berners-Lee, R. Fielding, and L. Masinter. Uniform resource identifier (URI): Generic syntax. Request for Comments RFC 3986, Internet Engineering Task Force, January 2005. URL <https://www.rfc-editor.org/rfc/rfc3986>.
- Mingyue Cheng, Jie Ouyang, Shuo Yu, Ruirao Yan, Yucong Luo, Zirui Liu, Daoyu Wang, Qi Liu, and Enhong Chen. Agent-rl: Training powerful llm agents with end-to-end reinforcement learning, 2025. URL <https://arxiv.org/abs/2511.14460>.
- Yize Cheng, Arshia Soltani Moakhar, Chenrui Fan, Parsa Hosseini, Kazem Faghieh, Zahra Sodagar, Wenxiao Wang, and Soheil Feizi. Your llm agents are temporally blind: The misalignment between tool use decisions and human time perception, 2026. URL <https://arxiv.org/abs/2510.23853>.
- João Vitor de Carvalho Silva and Douglas G. Macharet. Can llm agents solve collaborative tasks? a study on urgency-aware planning and coordination, 2025. URL <https://arxiv.org/abs/2508.14635>.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Commun. ACM*, 9(3):143–155, March 1966. ISSN 0001-0782. doi: 10.1145/365230.365252. URL <https://doi.org/10.1145/365230.365252>.
- R. Fielding and J. Reschke. Hypertext transfer protocol (HTTP/1.1): Conditional requests. Request for Comments RFC 7232, Internet Engineering Task Force, June 2014. URL <https://www.rfc-editor.org/rfc/rfc7232>.
- R. Fielding, M. Nottingham, and J. Reschke. HTTP semantics. Request for Comments RFC 9110, Internet Engineering Task Force, June 2022. URL <https://www.rfc-editor.org/rfc/rfc9110>.
- Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259, New York, NY, USA, 1987. Association for Computing Machinery. doi: 10.1145/38713.38742. URL <https://dl.acm.org/doi/10.1145/38713.38742>.
- Gemini Team, Google DeepMind. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025. URL <https://arxiv.org/abs/2507.06261>.
- Gemma Team. Gemma 4: Model card, 2025. URL https://ai.google.dev/gemma/docs/core/model_card_4.
- Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, January 2026. URL <https://github.com/laude-institute/harbor>.
- D. Hardt. The OAuth 2.0 authorization framework. Request for Comments RFC 6749, Internet Engineering Task Force, October 2012. URL <https://www.rfc-editor.org/rfc/rfc6749>.
- Yifeng He, Ethan Wang, Yuyang Rong, Zifei Cheng, and Hao Chen. Security of ai agents. In *2025 IEEE/ACM International Workshop on Responsible AI Engineering (RAIE)*, pages 45–52, 2025. doi: 10.1109/RAIE66699.2025.00013.

- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- M. Jones and D. Hardt. The OAuth 2.0 authorization framework: Bearer token usage. Request for Comments RFC 6750, Internet Engineering Task Force, October 2012. URL <https://www.rfc-editor.org/rfc/rfc6750>.
- Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020. URL <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. SkillsBench: Benchmarking how well agent skills work across diverse tasks, 2026. URL <https://arxiv.org/abs/2602.12670>.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: iterative refinement with self-feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NeurIPS '23*, 2023. URL <https://arxiv.org/abs/2303.17651>.
- Ian R. McKenzie, Alexander Lyzhov, Michael Pieler, Alicia Parrish, Aaron Mueller, Ameya Prabhu, Euan McLean, Xudong Shen, Joe Cavanagh, Andrew Gritsevskiy, Derik Kauffman, Aaron Kirtland, Zhengping Zhou, Yuhui Zhang, Sicong Huang, The Floating Droid, Tom Tseng, Daniel Wurgaft, Max Weiss, Alexis Ross, Gabriel Recchia, Alisa Liu, Jiacheng Liu, Tomasz Korbak, Najoung Kim, Samuel R. Bowman, and Ethan Perez. Inverse scaling: When bigger isn't better. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=DwgRm72GQF>. Featured Certification.
- Mike A Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Kwesi Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha

- Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Jenia Jitsev, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- Meta AI. Llama 3.3: Model card, 2024. URL <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>.
- Microsoft. Phi-4: Technical report, 2024. URL <https://huggingface.co/microsoft/phi-4>.
- MiniMax. MiniMax-M2 family: Model card, 2025. URL <https://huggingface.co/MiniMaxAI/MiniMax-M2>.
- Mistral AI. Mistral-3 series: Model cards, 2025a. URL <https://mistral.ai/news/mistral-3>.
- Mistral AI. Mistral-Small-4: Model card, 2025b. URL <https://mistral.ai/news/mistral-small-4>.
- NIST. Secure hash standard (SHS). Federal Information Processing Standards Publication FIPS PUB 180-4, U.S. Department of Commerce, August 2015. URL <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>.
- M. Nottingham and R. Fielding. Additional HTTP status codes. Request for Comments RFC 6585, Internet Engineering Task Force, April 2012. URL <https://www.rfc-editor.org/rfc/rfc6585>.
- OpenAI. GPT-4o: System card, 2024. URL <https://openai.com/index/gpt-4o-system-card/>. Pinned identifier: gpt-4o-2024-08-06.
- OpenAI. GPT-5: System card, 2025a. URL <https://openai.com/index/gpt-5-system-card/>.
- OpenAI. GPT-5.1: A smarter, more conversational ChatGPT, November 2025b. URL <https://openai.com/index/gpt-5-1/>.
- OpenAI. Introducing GPT-5.1 for developers, November 2025c. URL <https://openai.com/index/gpt-5-1-for-developers/>.
- OpenAI. Update to GPT-5 system card: GPT-5.2, 2026a. URL <https://openai.com/index/gpt-5-system-card-update-gpt-5-2/>.
- OpenAI. Introducing GPT-5.5, April 2026b. URL <https://openai.com/index/introducing-gpt-5-5/>. First fully retrained base model since GPT-4.5; GPT-5.x (5.0–5.4) releases were post-training updates on a shared pretrained base.
- Pei-Chi Pan, Yingbin Liang, and Sen Lin. Reward modeling for reinforcement learning-based llm reasoning: Design, challenges, and evaluation, 2026. URL <https://arxiv.org/abs/2602.09305>.
- R. Polli and L. Pardue. Digest fields. Request for Comments RFC 9530, Internet Engineering Task Force, February 2024. URL <https://www.rfc-editor.org/rfc/rfc9530>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- Qwen Team. Qwen 2.5: Technical report, 2024. URL <https://arxiv.org/abs/2412.15115>.

- Qwen Team. Qwen 3.5: Model card, 2025. URL <https://huggingface.co/Qwen/Qwen3.5-35B-A3B>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Yacmpz84TH>.
- Neil K. R. Sehgal, Sharath Chandra Guntuku, and Lyle Ungar. Real-time deadlines reveal temporal awareness failures in llm strategic dialogues, 2026. URL <https://arxiv.org/abs/2601.13206>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NeurIPS '23, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856. URL <https://openreview.net/forum?id=yzkSU5zdWd>. Survey Certification.
- Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. Travelplanner: a benchmark for real-world planning with language agents. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 54590–54613. PMLR, 2024. URL <https://proceedings.mlr.press/v235/xie24j.html>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385. URL <https://arxiv.org/abs/2405.15793>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R Narasimhan. τ -bench: A benchmark for Tool-Agent-User interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.
- Yuxuan Zhang, Yubo Wang, Yipeng Zhu, Penghui Du, Junwen Miao, Xuan Lu, Wendong Xu, Yunzhuo Hao, Songcheng Cai, Xiaochen Wang, Huaisong Zhang, Xian Wu, Yi Lu, Minyi Lei, Kai Zou, Huifeng Yin, Ping Nie, Liang Chen, Dongfu Jiang, Wenhui Chen, and Kelsey R. Allen. Clawbench: Can ai agents complete everyday online tasks?, 2026. URL <https://arxiv.org/abs/2604.08523>.
- Huaixiu Steven Zheng, Swaroop Mishra, Hugh Zhang, Xinyun Chen, Minmin Chen, Azade Nova, Le Hou, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, and Denny Zhou. Natural plan: Benchmarking llms on natural language planning. 2024. URL <https://arxiv.org/abs/2406.04520>.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=oKn9c6ytLx>.

Appendix

Table of Contents

A	Discussion	16
B	Limitations and Broader Impact	16
C	Full Benchmark Landscape	16
D	Full Task Catalog	17
E	Task File Anatomy	17
F	Contract Abstraction: Implementation	18
G	Episode Result Schema	19
H	Two Illustrative Tasks	19
I	Proof Sketch: Orthogonality of Validity and Integrity	19
J	Production Path Mutations	19
K	Severity-Weighted Failure-Label Mapping	20
L	Run-to-Run Reproducibility and Episode Budgets	20
M	Compute Resources and Pinned Model Checkpoints	21
N	Full Master Leaderboard	22
O	Base vs. Instruct: Post-Training Lifts the Floor	24
P	V-Shape Regression: Detailed Decomposition	24
Q	Per-Task Heatmaps (All 33 Tasks)	24
R	Per-Task Breakdown (Numerical)	25
S	Per-Label Retry Breakdown	25
T	Full Failure-Mode Profiles (All Models)	25

A Discussion

From findings to framework requirements. The three results in Section 4 jointly characterize observation-contract compliance as a fragile, post-training-driven inhibition capability: emergence is family-specific (Section 4.2), can regress at constant parameter count (Section 4.3), and admits actionable retry-time correction (Section 4.4). For agent framework designers, the operational consequence is that mitigations cannot be left to the next model upgrade. Four levers follow directly from the failure modes CONTRACTBENCH surfaces: (1) make artifact TTLs explicit and machine-readable (not buried in HTTP headers); (2) provide handle-based artifact storage to eliminate in-context byte mutation; (3) implement back-off middleware so that aggressive retry behavior cannot exhaust rate-limit quotas; (4) treat the structured failure label as a deployable corrective signal — inject it as a coaching note on retry (validated in Section 4.4: +7.1 pp gap between correct- and wrong-label coaching), and use it as a candidate process-level reward for downstream RL post-training [Cheng et al., 2025, Pan et al., 2026]. The label is unique among process signals because it is dense (per-failure semantic detail), automatically generated by the deterministic validator (no human annotation required), and actionable at retry time without retraining — in contrast to model-generated self-critique [Shinn et al., 2023, Madaan et al., 2023] or human-curated process-reward datasets like PRM800K.

Hard limits remain architectural. Some failures are not addressable by scale or by post-training alone. The architectural universality is striking: the best open-source MoE (Qwen3.5-397B-A17B at 70.7 %) lands within 7 pp of the best frontier model (Claude Opus 4.6 at 77.8 %), while the reasoning-trained DeepSeek-R1 (671 B / 37 B-active) collapses to 0 % — chain-of-thought training does not rescue contract compliance. Combined with the universally-failing tasks identified in Section 4.4 (notably `multi-turn-recall`, where every model in our cohort scores 0), these limits suggest that further progress will require architectural changes (*e.g.*, handle-based deferred binding [Dennis and Van Horn, 1966]) rather than larger or differently-tuned LLMs.

B Limitations and Broader Impact

Limitations. (1) The virtual clock abstracts away real-world timing challenges (network latency, API variability). This abstraction trades realism for reproducibility; production deployments should validate against real-time conditions. (2) We do not provide a human baseline. Each task ships with a deterministic reference solution (item 3.1) that confirms in-principle solvability, but a small human study would quantify the human-LLM gap explicitly. (3) Frontier proprietary models (Claude, GPT, Gemini) cannot be placed on the parameter-axis scaling figure because their parameter counts are not disclosed; their results appear in the leaderboard (Table 10) but not in Figure 3.

Broader impact. CONTRACTBENCH provides a deployable diagnostic for an under-tested capability: (1) operators can detect post-training regressions on contract-respecting behavior that general-capability benchmarks would miss (Section 4.3); (2) the structured failure taxonomy enables label-aware retry as a low-cost mitigation deployable today (Section 4.4). We do not foresee negative societal impacts: the benchmark evaluates agent robustness, not new capabilities.

C Full Benchmark Landscape

Appendix Table 5 reproduces the full 11-benchmark landscape comparison referenced in Section 1 (Table 1 in the main body lists a representative subset of seven). Column conventions are identical: “Validity” = temporal constraints (TTLs, rate limits, version conflicts), “Integrity” = byte-level artifact preservation, “Programmatic” = fully scriptable evaluation with no human raters or LLM-as-judge.

Table 5: Full agent benchmark landscape across 11 published benchmarks plus CONTRACTBENCH. Compact subset reproduced in Table 1.

Benchmark	#Tasks	Validity	Integrity	Programmatic
WebArena [Zhou et al., 2024]	812	✗	✗	✓
AgentBench [Liu et al., 2024]	1014	✗	✗	✓
TravelPlanner [Xie et al., 2024]	1225	✗	✗	Partial
SWE-bench [Jimenez et al., 2024]	2294	✗	✗	✓
NATURAL PLAN [Zheng et al., 2024]	3600	Partial	✗	✓
ToolBench [Qin et al., 2024]	16 464	✗	✗	Partial
TicToc [Cheng et al., 2026]	700+	Partial	✗	✓
TPS-Bench [Sehgal et al., 2026]	200	Partial	✗	✓
REALM-Bench [de Carvalho Silva and Macharet, 2025]	14	Partial	✗	✓
τ -bench [Yao et al., 2025]	165	✗	✗	✓
Terminal-Bench [Merrill et al., 2026]	80	✗	✗	✓
CONTRACTBENCH (Ours)	33	✓	✓	✓

D Full Task Catalog

E Task File Anatomy

Figure 5 reproduces the four files that make up a single CONTRACTBENCH task, using harbor/tasks/presigned-url-download/ as a representative example. The prose summary is in Table 3.1.

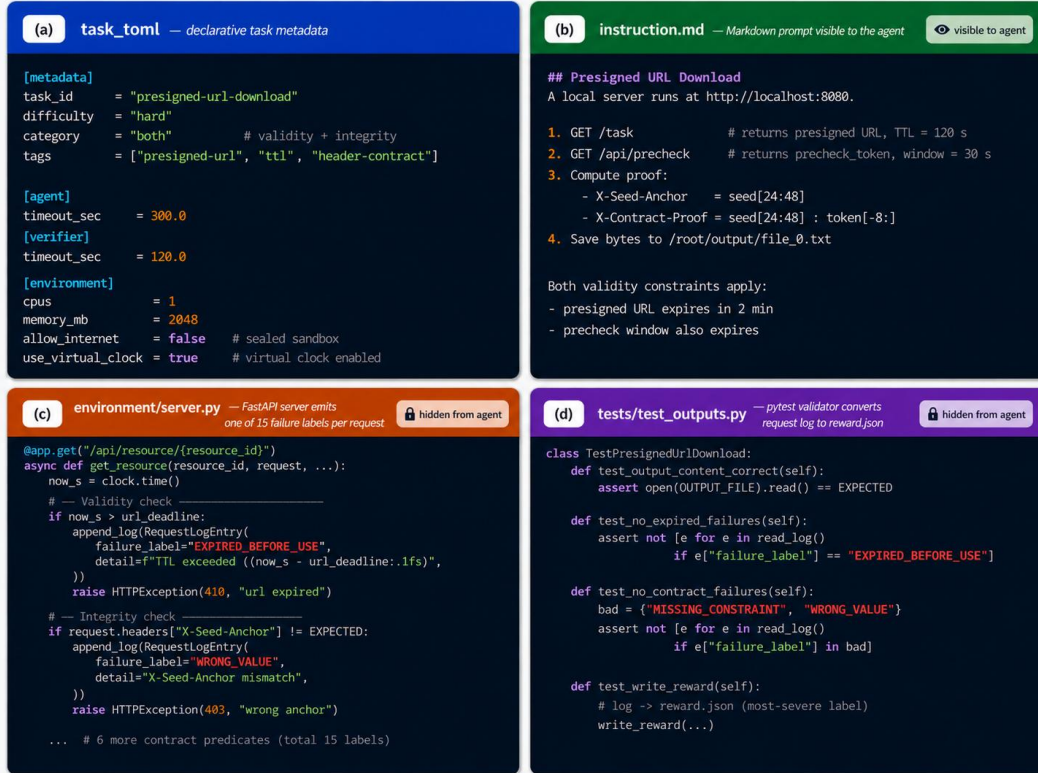


Figure 5: Anatomy of a CONTRACTBENCH task. Each task is four files: (a) TOML metadata, (b) Markdown instruction (agent-visible), (c) FastAPI server emitting failure labels over a virtual clock, (d) pytest validator producing the canonical reward.json. The agent sees only (b) and the server's HTTP responses; (a) and (d) are hidden, eliminating test-set leakage and LLM-as-judge ambiguity.

Table 6: Complete task catalog for CONTRACTBENCH (33 tasks).

Task ID	Difficulty	Quadrant	Contract Pattern
scheduled-maintenance	Medium	Q2	Timing/Backoff
api-rate-limit-patience	Hard	Q2	Timing/Backoff
token-refresh-workflow	Hard	Q2	Timing/Backoff
presigned-url-download	Hard	Q2	Timing/Backoff
csrf-form-submit	Hard	Q3	Byte-exact
url-trap-ellipsis	Hard	Q3	Byte-exact
long-token-handling	Hard	Q3	Byte-exact
extreme-url-length	Hard	Q3	Byte-exact
presigned-url-integrity	Hard	Q4	Signed requests
multi-token-workflow	Hard	Q4	Signed requests
multi-resource-priority	Hard	Q4	Resource mgmt
scattered-url-assembly	Hard	Q4	State chains
cumulative-hash-chain	Hard	Q4	State chains
multi-turn-recall	Hard	Q4	State chains
constraint-overload-protocol	Hard	Q4	Signed requests
adversarial-shortcut-injection	Hard	Q4	Signed requests
retry-backoff-compliance	Hard	Q2	Timing/Backoff
webhook-hmac-verify	V. Hard	Q3	Byte-exact
basic-oauth-token	Hard	Q4	OAuth/Auth
oauth-authorization-code	V. Hard	Q4	OAuth/Auth
oauth-pkce-with-rotation	Extreme	Q4	OAuth/Auth
api-key-rotation	V. Hard	Q4	OAuth/Auth
session-cookie-chain	V. Hard	Q4	OAuth/Auth
signed-request-canonicalization	V. Hard	Q4	Signed requests
certificate-pinning-handshake	Extreme	Q4	Signed requests
cursor-pagination-integrity	V. Hard	Q4	State chains
content-negotiation-chain	V. Hard	Q4	State chains
etag-conditional-get	Hard	Q4	Resource mgmt
idempotency-key-retry	V. Hard	Q4	Resource mgmt
distributed-lock-acquire	Extreme	Q4	Resource mgmt
multi-service-saga	Extreme	Q4	Multi-service
event-sourced-consistency	Extreme	Q4	Multi-service
cascading-token-revocation	V. Hard	Q4	Multi-service
Total			33

F Contract Abstraction: Implementation

```

@dataclass
class Contract:
    contract_id: str
    issued_at: float          # virtual clock time
    expires_at: float         # validity deadline
    expected_bytes_hash: str  # SHA-256 of canonical value
    resource_metadata: dict

    def validate_validity(self, current_time: float) -> bool:
        return current_time < self.expires_at

    def validate_integrity(self, submitted_value: str) -> bool:
        return sha256(submitted_value.encode()).hexdigest() \
            == self.expected_bytes_hash

    def validate(self, current_time, submitted_value) -> FailureLabel:
        if not self.validate_validity(current_time):
            return FailureLabel.EXPIRED_BEFORE_USE
        if not self.validate_integrity(submitted_value):

```

```

        return FailureLabel.MUTATED_TOKEN
    return FailureLabel.SUCCESS

```

G Episode Result Schema

Each episode produces a JSON object with the following fields:

```

{
  "task_id":          "multi-token-workflow",
  "category":         "both",
  "seed":             7,
  "agent":            "naive",
  "success":          false,
  "failure_label":    "EXPIRED_BEFORE_USE",
  "failure_detail": {
    "resource_id":    "file_B",
    "ttl_seconds":    15,
    "time_of_attempt": 42.3,
    "expired_by_seconds": 27.3
  },
  "steps":            4,
  "tool_calls":        3,
  "virtual_time_elapsed": 42.3,
  "trace_hash":       "a1b2c3..."
}

```

H Two Illustrative Tasks

Validity-heavy. The agent receives three presigned download URLs with TTLs of 10s, 30s, and 60s (virtual time); each download takes 8s. A naive agent that processes URLs in listed order finds the first URL expired by the time it attempts the second download. A validity-aware agent reorders by urgency (shortest TTL first) and succeeds on all three.

Integrity-heavy. A tool returns a presigned S3 URL with a 256-character query string containing an HMAC signature; the agent’s tool-call interface has a 200-character input limit. A direct-pass agent submits the truncated URL, which fails HMAC validation (TOOL_INPUT_TOO_LONG). A handle-based agent stores the URL server-side and submits a short capability handle (@HANDLE:ur1_1), which is resolved to the full URL at execution time.

I Proof Sketch: Orthogonality of Validity and Integrity

We restate Proposition 1: for any contract C with non-empty validity window $W(C)$ and non-trivial integrity predicate π , all four cells of the 2×2 partition $\{V, \neg V\} \times \{I, \neg I\}$ over submissions are non-empty.

Proof sketch. Pick any $t_{in} \in W(C)$ (non-empty by hypothesis) and any $t_{out} \notin W(C)$. Pick $o_{ok} = o$ so $\pi(o_{ok}) = 1$, and pick o_{bad} with $\pi(o_{bad}) = 0$ (such an o_{bad} exists because π is non-trivial). The four submissions

$$(o_{ok}, t_{in}), (o_{ok}, t_{out}), (o_{bad}, t_{in}), (o_{bad}, t_{out})$$

respectively populate the cells (V, I) , $(\neg V, I)$, $(V, \neg I)$, $(\neg V, \neg I)$. Hence none is empty, and fixing one axis does not constrain the other. $\square$

J Production Path Mutations

In production, the path from API response, through the agent’s context window, into the next tool call is not byte-preserving. LangChain truncates tool inputs past its token limit. HTTP libraries quietly

re-encode percent-escaped URLs. Middleware re-sorts query parameters before signing. Rendered link text disagrees with the underlying href. None of these are corruptions we introduce; each one is a documented behavior of widely deployed agent infrastructure. The agent has no way to see any of it from the context: it reads a plausible string, sends it, and the server validates different bytes. CONTRACTBENCH reproduces five of these naturally-occurring mutations as deterministic toggles so each cause can be isolated:

- **Truncation.** The tool-call interface clips an artifact past its input limit. The agent submits a partial HMAC, and the server returns `TOOL_INPUT_TOO_LONG`.
- **Line-wrap insertion.** A long token gets soft-wrapped with `\n`. The agent has to reassemble the original string before submitting; if it misses, the server returns `MUTATED_TOKEN`.
- **URL re-encoding.** A percent-encoded `%2F` is normalized to `/`. Any HMAC computed over the original encoding no longer matches, and the server returns `MUTATED_TOKEN`.
- **Query reorder.** Middleware re-sorts query parameters before the request leaves the agent. AWS SigV4 and Stripe HMAC are computed over an exact parameter order, so the server returns `SIGNATURE_MISMATCH`.
- **UI trap.** Rendered text (*e.g.*, the link label "Download") does not match the underlying href. An agent that submits the visible string instead of the byte value triggers `WRONG_VALUE`.

K Severity-Weighted Failure-Label Mapping

The full mapping from CONTRACTBENCH’s 15 failure labels to severity-weighted capability deficits is given in Appendix Table 7. The weights serve two operational roles in this paper: (i) most-severe-label aggregation when an episode emits multiple labels, which determines the primary label written to `reward.json` and visualized in Figure 9; (ii) test-time retry coaching — the most-severe label is the corrective hint injected at inference in the predictive-validity experiment of Table 4, with no model retraining required. Using the same severity-weighted mapping as a process-level reward signal during RL post-training is a natural next step [Cheng et al., 2025, Pan et al., 2026] that we leave to future work.

Table 7: Severity-weighted mapping from CONTRACTBENCH failure labels to capability deficits.

Failure Label	Severity	Capability Deficit
SUCCESS	+1.0	Reinforce correct behavior
EXPIRED_BEFORE_USE	−1.0	Temporal awareness, deadline planning
MUTATED_TOKEN	−1.0	Byte-level preservation
SHORTCUT_TAKEN	−0.9	Resist adversarial shortcuts
WRONG_VALUE	−0.8	Constraint satisfaction
MISSING_CONSTRAINT	−0.7	Protocol completeness
RATE_LIMITED	−0.5	Patience and backoff strategy
OTHER	−0.3	General task completion

L Run-to-Run Reproducibility and Episode Budgets

Episode budgets. Each task fixes a step budget and a virtual-time budget in its TOML header. Episodes that exceed either budget, raise an unrecoverable tool error, or are rate-limited terminate immediately, receive the appropriate failure label, and count as failures in the $n=99$ per-model total (Section 3.2), consistent with the conventions of Terminal-Bench [Merrill et al., 2026] and τ -bench [Yao et al., 2025].

Determinism budget. Four design choices drive run-to-run determinism: (i) the virtual clock eliminates real-time dependencies; (ii) seeded randomness controls every stochastic element; (iii) each episode records a SHA-256 trace hash; (iv) all LLM experiments use temperature 0 with pinned model IDs and logged API parameters. We use $k=3$ rather than the $k \geq 5$ of Terminal-Bench because empirically the bulk of cells are bit-identical at $k=3$ and the variance is concentrated on a small set of timing-sensitive tasks, characterized below.

Empirical determinism. The reproducibility claim of Section 3.2 is grounded in the released corpus ($n=2,259$ episodes across 25 models with ≥ 2 runs per (model, task) cell, totalling 733 such cells). **86.6% of (model, task) pairs are perfectly deterministic across $k=3$ runs**; the residual 13.4% concentrates on a small set of timing-sensitive tasks (Appendix Table 8, Appendix Figure 6). The variance is not measurement noise: in every variable cell, the variance comes from genuine stochasticity in the agent’s strategy under tight TTL or rate-limit pressure, not from validator non-determinism (the virtual clock + SHA-256 trace hash hold all else fixed).

Table 8: Top variance-driving tasks across all evaluated models, ordered by the number of distinct models whose three runs disagree. Eight tasks account for the majority of run-to-run variance; the remaining 25 tasks are essentially deterministic.

Task	# models w/ var.	Why variance arises
oauth-pkce-with-rotation	7	Token rotation interleaves with rate-limit windows
webhook-hmac-verify	6	Stochastic delivery retry + HMAC re-derivation
multi-token-workflow	6	Multi-step token-swap; one mid-flow misstep cascades
oauth-authorization-code	6	State parameter handling under concurrent attempts
presigned-url-download	5	Tight TTL races first vs. second download attempt
api-rate-limit-patience	5	429-aware retry behavior is timing-sensitive
cumulative-hash-chain	4	Long-context drift in hash-chain reconstruction
long-token-handling	4	Truncation depends on the model’s tool-call buffer

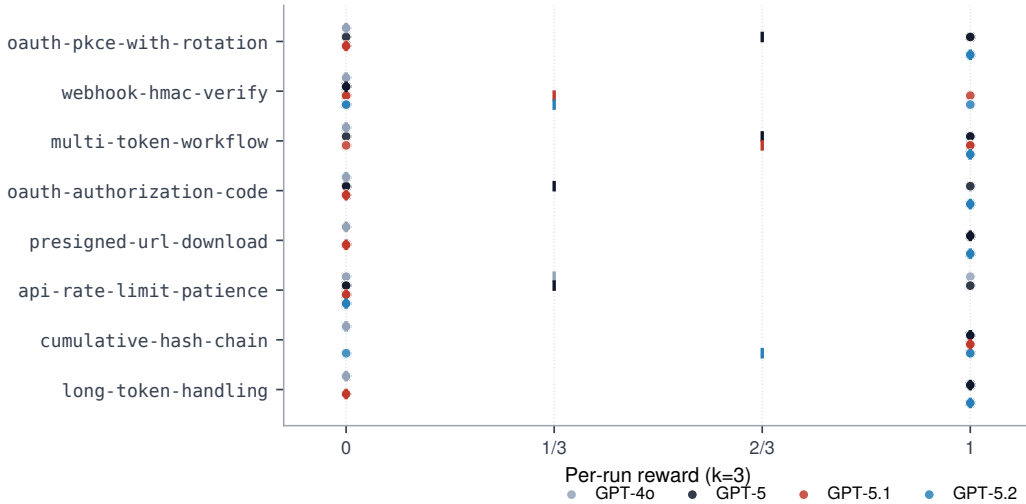


Figure 6: **Run-to-run variability on the five most variance-prone tasks for the GPT-5 series.** Each dot is a single run’s reward; the vertical bar is the per-(model, task) mean. Variance is bounded ($k=3$ rewards $\in \{0, \frac{1}{3}, \frac{2}{3}, 1\}$) and concentrates on tasks where adaptive timing decisions are pivotal; on most other tasks the same three runs collapse onto a single point.

M Compute Resources and Pinned Model Checkpoints

CONTRACTBENCH evaluation runs against two backend types: external LLM APIs (proprietary frontier models and many open-source models hosted by inference providers) and a local vLLM [Kwon et al., 2023] server (for the smaller open-source variants we hosted ourselves). All experiments use temperature 0; per-episode wall-clock budget is 600 s (Section 4); per-model episode count is $k=3$ rollouts $\times$ 33 tasks = $n=99$ episodes.

API-served models. We use pinned model identifiers to keep the evaluation reproducible against a moving frontier. The pinned IDs at evaluation time are listed in Appendix Table 9; they are also recorded inside every `reward.json` record in the released corpus, so any future rerun can be checked for drift. API providers used: Anthropic (Claude family), OpenAI (GPT family), Google AI Studio

(Gemini family), Hugging Face Inference (Qwen 3.5 family + Qwen 2.5 + the Mistral/Minstral series), Together AI (Llama-3.3-70B, MiniMax-M2.5, DeepSeek-R1), and OpenRouter (Gemma 4 series, MiniMax variants, Mistral-Small-4). Aggregate API spend across the full 38-row evaluation was approximately \$200 USD. Wall-clock for one full leaderboard pass is roughly 30 hours when parallelized across providers and roughly 5–7 days serialized.

Local vLLM-served models. The smaller open-source variants we did not have hosted-API access to (Qwen3.5-9B / 4B / Base variants, Phi-4 14 B Base) were served locally via vLLM 0.7. Hardware: a single workstation with $1 \times$ NVIDIA A100 (80 GB) for the 9 B and 14 B models, with model weights pulled from Hugging Face. Per-episode latency under vLLM: 5–40 s depending on the model and the task’s tool-call depth; per-model wall-clock for the full 99-episode pass: 15–90 minutes. No fine-tuning, no LoRA, no training; the local stack is inference-only.

Aggregation and figure scripts. The aggregation scripts (experiments/scripts/aggregate_failure_labels.py, the figure generators under paper/scripts/) run on a single CPU in under five minutes per task. Total local CPU usage for all paper figures is approximately 10 minutes.

Preliminary and abandoned runs. Beyond the reported runs, we ran approximately $1.5 \times$ the reported episodes across (a) early task iterations that were superseded before the 33-task release was frozen, and (b) a small set of debug runs to diagnose timing-sensitive variance on the eight tasks documented in Appendix Table 8. None of these are included in the leaderboard or the released corpus.

Table 9: Pinned model identifiers and provenance for every model in the leaderboard. “Provenance” cites the model card or technical report for the asset.

Model	Provider / Pinned ID	Provenance
<i>Frontier proprietary</i>		
Claude Opus 4.6	Anthropic / claude-opus-4-6	[Anthropic, 2025b]
Claude Sonnet 4.5	Anthropic / claude-sonnet-4-5-20250929	[Anthropic, 2025a]
GPT-5.2	OpenAI / gpt-5.2	[OpenAI, 2026a]
GPT-5.1	OpenAI / gpt-5.1	[OpenAI, 2025b]
GPT-5	OpenAI / gpt-5	[OpenAI, 2025a]
GPT-4o	OpenAI / gpt-4o-2024-08-06	[OpenAI, 2024]
Gemini 2.5 Pro	Google AI Studio / gemini-2.5-pro	[Gemini Team, Google DeepMind, 2025]
Gemini 2.5 Flash	Google AI Studio / gemini-2.5-flash	[Gemini Team, Google DeepMind, 2025]
<i>Open-source SOTA</i>		
Qwen3.5-397B-A17B	Hugging Face / Qwen/Qwen3.5-397B-A17B	[Qwen Team, 2025]
Qwen3.5-27B	Hugging Face / Qwen/Qwen3.5-27B	[Qwen Team, 2025]
Qwen3.5-9B	Hugging Face / Qwen/Qwen3.5-9B	[Qwen Team, 2025]
Qwen2.5-72B-Instruct	Hugging Face / Qwen/Qwen2.5-72B-Instruct	[Qwen Team, 2024]
MiniMax-M2.5	Together / MiniMaxAI/MiniMax-M2.5	[MiniMax, 2025]
MiniMax-M2.1	Hugging Face / MiniMaxAI/MiniMax-M2.1	[MiniMax, 2025]
MiniMax-M2	Hugging Face / MiniMaxAI/MiniMax-M2	[MiniMax, 2025]
Mistral-Small-4	OpenRouter / mistralai/mistral-small-2603	[Mistral AI, 2025b]
Minstral-3-14B	OpenRouter / mistralai/ministral-14b-2512	[Mistral AI, 2025a]
Minstral-3-8B	OpenRouter / mistralai/ministral-8b-2512	[Mistral AI, 2025a]
Minstral-3-3B	OpenRouter / mistralai/ministral-3b-2512	[Mistral AI, 2025a]
Gemma-4-26B-A4B	OpenRouter / google/gemma-4-26b-a4b-it	[Gemma Team, 2025]
Gemma-4-31B	Hugging Face / google/gemma-4-31b-it	[Gemma Team, 2025]
Gemma-4-E4B	Ollama / gemma-4-e4b	[Gemma Team, 2025]
Llama-3.3-70B	Together / meta-llama/Llama-3.3-70B-Instruct-Turbo	[Meta AI, 2024]
DeepSeek-R1	Together / deepseek-ai/DeepSeek-R1	[DeepSeek-AI, 2025]
Phi-4 14B	vLLM / microsoft/phi-4	[Microsoft, 2024]

N Full Master Leaderboard

Table 3 in the main body shows the frontier-proprietary subset that motivates RQ1. Appendix Table 10 below is the full master leaderboard across all 38 model variants $\times$ 33 tasks ($k=3$, $n=99$ episodes per row), grouped into frontier proprietary, open-source SOTA, and sub-cliff blocks. Within each block, rows are grouped by family and ordered by SR. The Size and Type / Provider columns record provider/infrastructure metadata; Status (✓) marks complete coverage.

Table 10: Master leaderboard: 38 model variants $\times$ 33 tasks ($k=3$, $n=99$). Within each block, rows are grouped by family and ordered by SR. Status: ✓complete.

Model	Size	Type / Provider	Eps	Pass	SR%	Status
<i>Frontier proprietary</i>						
Claude Opus 4.6	N/A	Instruct / Anthropic	99	77	77.8	✓
Claude Sonnet 4.5	N/A	Instruct / Anthropic	99	69	69.7	✓
Gemini 2.5 Pro	N/A	Instruct / Google	99	51	51.5	✓
Gemini 2.5 Flash	N/A	Instruct / Google	99	19	19.2	✓
GPT-5.2	closed	Instruct / OpenAI	99	74	74.7	✓
GPT-5.1	closed	Instruct / OpenAI	99	48	48.5	✓
GPT-5	closed	Instruct / OpenAI	99	70	70.7	✓
GPT-4o	closed	Instruct / OpenAI	99	23	23.2	✓
<i>Open-source SOTA (Instruct/MoE)</i>						
Qwen3.5-397B-A17B (MoE)	397B/17B	Instruct / HF	99	70	70.7	✓
Qwen3.5-27B	27B	Instruct / HF	99	64	64.6	✓
Qwen3.5-9B	9B	Instruct / HF	99	56	56.6	✓
Qwen2.5-72B-Instruct	72B	Instruct / HF	99	23	23.2	✓
MiniMax-M2.5	N/A	Instruct / Together	99	62	62.6	✓
MiniMax-M2.1	N/A	Instruct / HF	99	60	60.6	✓
MiniMax-M2	N/A	Instruct / HF	99	53	53.5	✓
Mistral-Small-4 (MoE)	119B/6.5B	Instruct / Open-Router	99	42	42.4	✓
Ministral-3-14B	14B	Instruct / Open-Router	99	28	28.3	✓
Ministral-3-8B	8B	Instruct / Open-Router	99	19	19.2	✓
Ministral-3-3B	3B	Instruct / Open-Router	99	6	6.1	✓
Gemma-4-31B	31B	Instruct / Open-Router	99	37	37.4	✓
Gemma-4-26B-A4B (MoE)	26B/4B	Instruct / Open-Router	99	38	38.4	✓
Gemma-4-E4B	4B (eff.)	Instruct / Ollama	99	17	17.2	✓
Gemma-4-E2B	2B (eff.)	Instruct / Ollama	99	7	7.1	✓
Llama-3.3-70B-Instruct	70B	Instruct / Together	99	7	7.1	✓
<i>Sub-cliff (no contract-level emergence; all $\sim$ 0%)</i>						
Qwen3.5-9B	9B	Base / vllm	99	0	0.0	✓
Qwen3.5-4B (Instruct)	4B	Instruct / vllm	99	0	0.0	✓
Qwen3.5-4B (Base)	4B	Base / vllm	99	0	0.0	✓
Qwen2.5-72B (Base)	72B	Base / Featherless	99	0	0.0	✓
Qwen2.5-32B (Instruct)	32B	Instruct / Featherless	99	0	0.0	✓
Qwen2.5-32B (Base)	32B	Base / Featherless	99	0	0.0	✓
Qwen2.5-7B (Instruct)	7B	Instruct / Featherless	99	0	0.0	✓
Qwen2.5-7B (Base)	7B	Base / Featherless	99	0	0.0	✓
Qwen2.5-1.5B (Instruct)	1.5B	Instruct / Featherless	99	0	0.0	✓
Qwen2.5-1.5B (Base)	1.5B	Base / Featherless	99	0	0.0	✓
Phi-4 (vllm)	14B	Base / vllm	99	0	0.0	✓
DeepSeek-R1 (Thinking)	671B/37B	Thinking / Together	99	0	0.0	✓
Ministral-3-14B (Base)	14B	Base / vllm	99	0	0.0	✓
Ministral-3-8B (Base)	8B	Base / vllm	99	0	0.0	✓
Ministral-3-3B (Base)	3B	Base / vllm	99	0	0.0	✓

O Base vs. Instruct: Post-Training Lifts the Floor

Section 4.2 reports the within-family scaling cliff along the parameter axis of the Qwen 3.5 *Instruct* row. The orthogonal Base→Instruct axis of Figure 3 carries a separate, sharper signal that we summarize here for completeness.

Every Base variant we evaluated — Qwen 3.5 (4 B, 9 B), Qwen 2.5 (1.5 B, 7 B, 32 B, 72 B), Ministral-3 (3 B, 8 B, 14 B), Phi-4 14 B — scores 0 % on the full 33-task suite. Base models lack a chat template and tool-call format, so on a benchmark whose every task requires tool calls they emit ungrounded text and the validator records 100 % OTHER. The contrast with the Instruct row is the cleanest evidence in this paper that contract compliance is a property of *post-training*, not parameter count: at fixed parameters, reading down a column of Figure 3 isolates the Base→Instruct delta with everything else held constant. (Qwen only publishes Base checkpoints up to 9 B; the 27 B and 397 B-A17 B variants are Instruct-only.) The complementary post-training observation — that helpfulness-oriented updates can *erode* compliance at constant parameters — is the V-shape regression of Section 4.3.

P V-Shape Regression: Detailed Decomposition

Section 4.3 reports a structured, locatable post-training regression in the GPT-5 family. Here we record the numbers that back that claim. Per-family decomposition (Figure 4b) shows integrity collapses hardest from GPT-5 to GPT-5.1 (0.80 → 0.47, −0.33), versus −0.25 on validity and −0.20 on hybrid. At the label level (Figure 4c), GPT-5.1’s 51 failures are dominated by WRONG_VALUE (21) plus MUTATED_TOKEN (4) — byte-level integrity labels that had near-disappeared in GPT-5. GPT-5.2’s residual failures shift back toward temporal labels rather than reversing GPT-5.1’s integrity surge, which is why the recovery is described in the main body as a different point in the failure-mode space rather than a re-traversal of GPT-5’s path.

Q Per-Task Heatmaps (All 33 Tasks)

Appendix Figure 7 and Appendix Figure 8 give the per-task pass-rate heatmaps referenced in Section 4.4. Cells are mean reward across $k=3$ runs; tasks are family-grouped (Validity / Integrity / Hybrid). Sub-cliff models that score 0 on every task (DeepSeek-R1, Phi-4, Qwen3.5-9B/4B base, Qwen2.5-72B base) are omitted from Appendix Figure 8 for visual clarity — their leaderboard rows remain in Appendix Table 10.

Three universal frontiers. The heatmaps expose three frontiers that no model crosses: (1) **universal floor** — multi-turn-recall (preserving an 8 192-byte URL across conversation history) is 0.00 for every model, an architectural limit on context-window byte fidelity; (2) **intra-family inverse scaling** — on api-rate-limit-patience the most-capable models do *worse* (GPT-4o 0.80 → GPT-5.1/5.2 0.00; only Claude Opus 4.6 clears it), because aggressive retry exhausts rate-limit quotas instead of waiting; (3) **universal ceiling** — no model exceeds 77.8 % (Claude Opus 4.6) on the full suite. Frontiers (1) and (2) motivate the architectural-intervention pointer in Finding 5: the failures they describe are exactly the ones in-context label coaching does not recover.

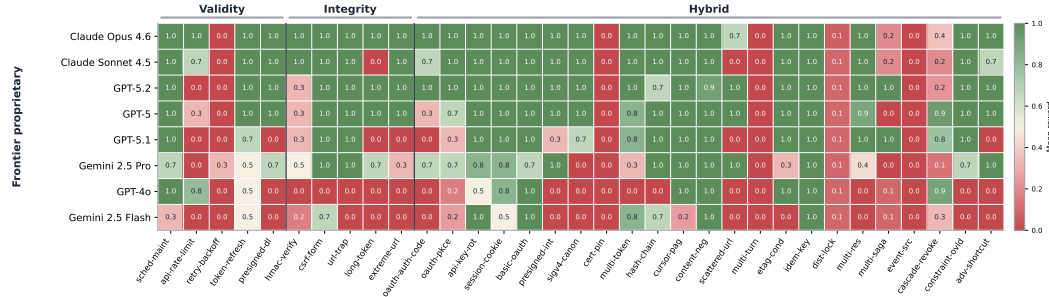


Figure 7: **Per-task pass-rate heatmap: frontier proprietary models × 33 tasks.** The Hybrid block reveals the universally-hard tasks (e.g., multi-turn-recall is red across every row).

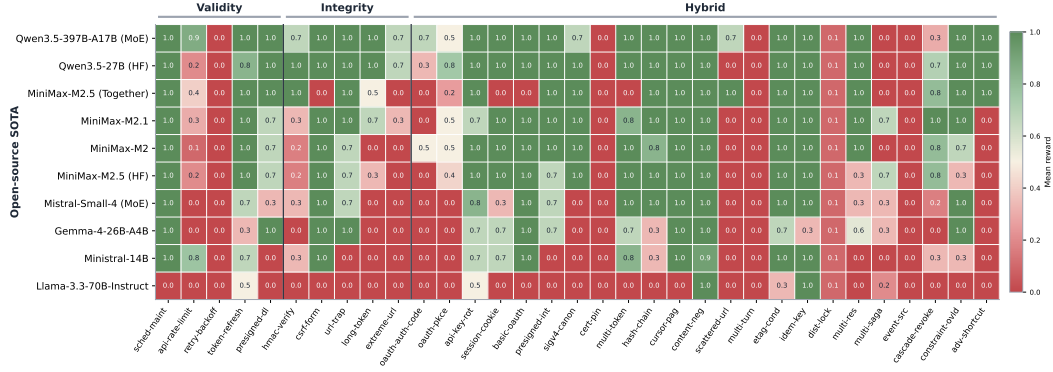


Figure 8: **Per-task pass-rate heatmap: open-source SOTA $\times$ 33 tasks.** Same color scale and family grouping as Appendix Figure 7.

R Per-Task Breakdown (Numerical)

Appendix Table 11 and Appendix Table 12 give the exact numerical per-task pass rates that the heatmaps in Appendix Figure 7 and Appendix Figure 8 visualize. Each cell is the mean reward across $k=3$ runs of one (model, task) pair on the full 33-task suite; **bold** marks the per-task best within each panel. Models without full $k=3$ coverage on every task at the time of writing are omitted (Mistral / Ministral, MiniMax-M2 / M2.1, Qwen 2.5 family, Gemini 2.5 Flash); their leaderboard rows remain in Appendix Table 10. The bottom row of each table is the unweighted 33-task mean per model.

S Per-Label Retry Breakdown

The +7.1 pp paired gap reported in Table 4 (main body Section 4.4) is not uniform across failure labels. Appendix Table 13 below decomposes the 42 paired GPT-5.1 episodes by the failure label assigned at first attempt, showing where retry-time coaching helps and where it does not. The headline asymmetry — recovery concentrates in integrity-style labels (WRONG_VALUE, MISSING_CONSTRAINT); temporal labels are unresponsive — and the RATE_LIMITED inversion are discussed in the main body.

T Full Failure-Mode Profiles (All Models)

Appendix Figure 9 arranges seven complete-coverage models from Appendix Table 10 as a capability-ladder pie grid (ordered by SR). Phi-4 14 B (sub-cliff) fails entirely with OTHER — a *pre-contract failure* where the agent gives up before reaching any contract predicate, not a taxonomy coverage gap. From there the dominant failure type shifts predictably with capability: integrity (WRONG_VALUE, red) dominates mid-tier and cliff-emerging models, then temporal (EXPIRED_BEFORE_USE, blue) takes over at the frontier (Sonnet 4.5, Opus 4.6); the open-source MoE ceiling (Qwen3.5-397B-A17B) sits at the same transition point. GPT-4o’s red-dominant profile is exactly what GPT-5.1 regresses toward in Section 4.3, motivating the GPT-5.1 test bed used in Section 4.4.

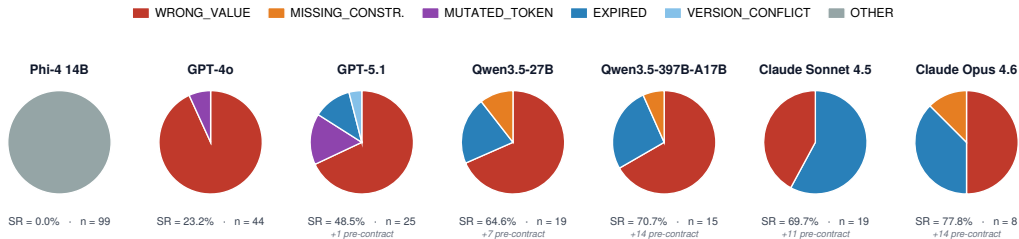


Figure 9: Capability ladder of failure-mode profiles, ordered by SR. The dominant failure type shifts gray (sub-cliff OTHER) $\rightarrow$ red (mid-tier integrity) $\rightarrow$ blue (frontier temporal).

Table 11: Per-task pass rate, frontier proprietary models ($k=3$, full 33-task suite).

Family	Task	GPT-4o	GPT-5	GPT-5.1	GPT-5.2	Op4.6	So4.5	G2.5Pro
Validity	api-rate-limit	0.33	0.33	0.00	0.00	1.00	0.67	0.00
	retry-backoff	0.00	0.00	0.00	0.00	0.00	0.00	0.33
	sched-maint	1.00	1.00	1.00	1.00	1.00	1.00	0.67
	token-refresh	0.00	1.00	0.33	1.00	1.00	1.00	0.33
Integrity	csrf-form	0.00	1.00	1.00	1.00	1.00	1.00	1.00
	extreme-url	0.00	1.00	0.00	1.00	1.00	1.00	0.33
	long-token	0.00	1.00	0.00	1.00	1.00	0.00	0.67
	url-trap	0.00	1.00	1.00	1.00	1.00	1.00	1.00
	webhook-hmac	0.00	0.00	0.33	0.33	1.00	1.00	0.33
Hybrid	adv-shortcut	0.00	1.00	0.00	1.00	1.00	0.67	1.00
	api-key-rot	0.00	1.00	1.00	1.00	1.00	1.00	0.67
	oauth-basic	1.00	1.00	1.00	1.00	1.00	1.00	0.67
	cascade-revoke	0.67	0.67	0.00	0.00	0.00	0.00	0.00
	cert-pinning	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	constr-overload	0.00	1.00	1.00	1.00	1.00	1.00	0.67
	content-neg	1.00	1.00	1.00	0.67	1.00	1.00	1.00
	hash-chain	0.00	1.00	1.00	0.67	1.00	1.00	1.00
	cursor-pag	1.00	1.00	1.00	1.00	1.00	1.00	1.00
	dist-lock	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	etag-cond	1.00	1.00	1.00	1.00	1.00	1.00	0.33
	event-source	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	idempotency	1.00	1.00	1.00	1.00	1.00	1.00	1.00
	multi-resource	0.00	0.67	1.00	1.00	1.00	1.00	0.00
	saga	0.00	0.00	1.00	1.00	0.00	0.00	0.00
	multi-token	0.00	0.67	0.67	1.00	1.00	1.00	0.33
	multi-turn-rec	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	oauth-code	0.00	0.33	0.00	1.00	1.00	0.67	0.67
	oauth-pkce	0.00	0.67	0.00	1.00	1.00	1.00	0.67
	presigned-dl	0.00	1.00	0.00	1.00	1.00	1.00	0.67
	presigned-int	0.00	1.00	0.33	1.00	1.00	1.00	1.00
	scattered-url	0.00	1.00	0.00	1.00	0.67	0.00	1.00
	session-cookie	0.67	1.00	1.00	1.00	1.00	1.00	0.67
	sig-canonical	0.00	1.00	0.33	1.00	1.00	1.00	0.00
33-task mean		0.23	0.71	0.48	0.75	0.78	0.70	0.52

Appendix Figure 10 below complements the pie-grid view with the full per-model stacked-bar profile across all 14 models with ≥ 60 episode coverage in our cohort, including the full GPT-5 series (4o / 5 / 5.1 / 5.2) and the open-source SOTA tier. The stacked-bar view exposes per-label proportions as horizontal bands that are easy to scan across models (e.g., the `WRONG_VALUE` band visibly grows in GPT-5.1 vs. GPT-5, then shrinks in GPT-5.2 — the V-shape of Section 4.3). The pie grid sacrifices that horizontal-scan property in exchange for showing the dominant-failure-type progression along the capability axis as a single Gestalt; the two figures answer different questions on the same underlying data.

Table 12: Per-task pass rate, open-source SOTA ($k=3$, full 33-task suite). Column abbreviations: Q3.5-397B = Qwen3.5-397B-A17B, Q3.5-27B = Qwen3.5-27B, M2.5 = MiniMax-M2.5, Gem4-26B = Gemma-4-26B-A4B, Llama70B = Llama-3.3-70B-Instruct, DS-R1 = DeepSeek-R1.

Family	Task	Q3.5-397B	Q3.5-27B	M2.5	Gem4-26B	Llama70B	DS-R1	Phi-4
Validity	api-rate-limit	0.67	0.00	0.00	0.00	0.00	0.00	0.00
	retry-backoff	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	sched-maint	1.00	1.00	1.00	1.00	0.00	0.00	0.00
	token-refresh	1.00	0.67	1.00	0.00	0.00	0.00	0.00
Integrity	csrf-form	1.00	1.00	0.67	1.00	0.00	0.00	0.00
	extreme-url	0.67	0.67	0.67	0.00	0.00	0.00	0.00
	long-token	1.00	0.00	0.33	0.00	0.00	0.00	0.00
	url-trap	1.00	1.00	1.00	1.00	0.00	0.00	0.00
	webhook-hmac	0.67	1.00	1.00	0.00	0.00	0.00	0.00
Hybrid	adv-shortcut	1.00	1.00	0.67	0.00	0.00	0.00	0.00
	api-key-rot	1.00	1.00	1.00	0.67	0.00	0.00	0.00
	oauth-basic	1.00	1.00	1.00	1.00	0.00	0.00	0.00
	cascade-revoke	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	cert-pinning	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	constr-overload	1.00	1.00	1.00	1.00	0.00	0.00	0.00
	content-neg	1.00	1.00	1.00	1.00	1.00	0.00	0.00
	hash-chain	1.00	1.00	1.00	0.33	0.00	0.00	0.00
	cursor-pag	1.00	1.00	1.00	1.00	0.00	0.00	0.00
	dist-lock	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	etag-cond	1.00	1.00	1.00	0.67	0.33	0.00	0.00
	event-source	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	idempotency	1.00	1.00	1.00	0.33	1.00	0.00	0.00
	multi-resource	1.00	1.00	1.00	0.33	0.00	0.00	0.00
	saga	0.00	0.00	0.00	0.33	0.00	0.00	0.00
	multi-token	1.00	1.00	0.33	0.67	0.00	0.00	0.00
	multi-turn-rec	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	oauth-code	0.67	0.33	0.00	0.00	0.00	0.00	0.00
	oauth-pkce	0.33	0.67	0.00	0.00	0.00	0.00	0.00
	presigned-dl	1.00	1.00	1.00	1.00	0.00	0.00	0.00
	presigned-int	1.00	1.00	1.00	0.67	0.00	0.00	0.00
	scattered-url	0.67	0.00	1.00	0.00	0.00	0.00	0.00
	session-cookie	1.00	0.00	1.00	0.67	0.00	0.00	0.00
	sig-canonical	0.67	1.00	1.00	0.00	0.00	0.00	0.00
33-task mean		0.71	0.62	0.63	0.38	0.07	0.00	0.00

Table 13: Per-label retry success on the $n=42$ paired GPT-5.1 failures. Cells show pass-count under each retry condition; the right column gives the Δ between correct- and wrong-label coaching.

Failure label	n	Naive	Correct-label	Wrong-label	Δ (correct – wrong)
WRONG_VALUE	19	3	4	2	+2
MISSING_CONSTRAINT	1	0	1	0	+1
EXPIRED_BEFORE_USE	9	2	2	1	+1
RATE_LIMITED	1	0	0	1	−1 (inversion)
other (rare labels)	12	1	1	1	0
Total	42	6	8	5	+3 pass-count, +7.1 pp rate

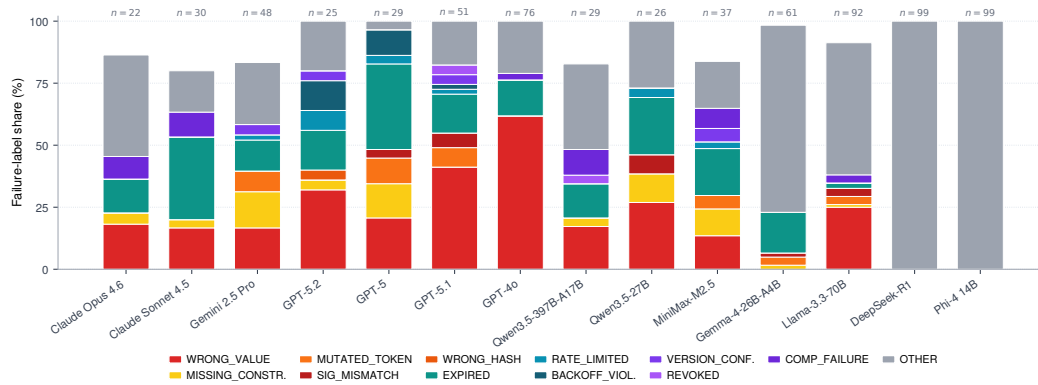


Figure 10: Full per-model failure-label profile (stacked bars, all 15 complete- or near-complete-coverage models). Each bar’s segments sum to the model’s total failed-episode count (printed as $n = \dots$ above the bar); segment heights are the proportion of failed episodes attributable to each label category. Companion view to the capability-ladder pie grid in Appendix Figure 9.